

Inteligência Artificial

Redes Madaline



JeffersonBMartins
Jeffersonbeethm
Por aqui desde maio de 2014
Segue 234 Tem 152 seguidores

+ ADICIONAR AMIGOS

Estatísticas

696 Dias seguidos	55278 Total de XP
Safira Divisão	3 Pódios

Prof. Dr. Jefferson Beethoven Martins
IFTM Campus UPT – 03/03/2026



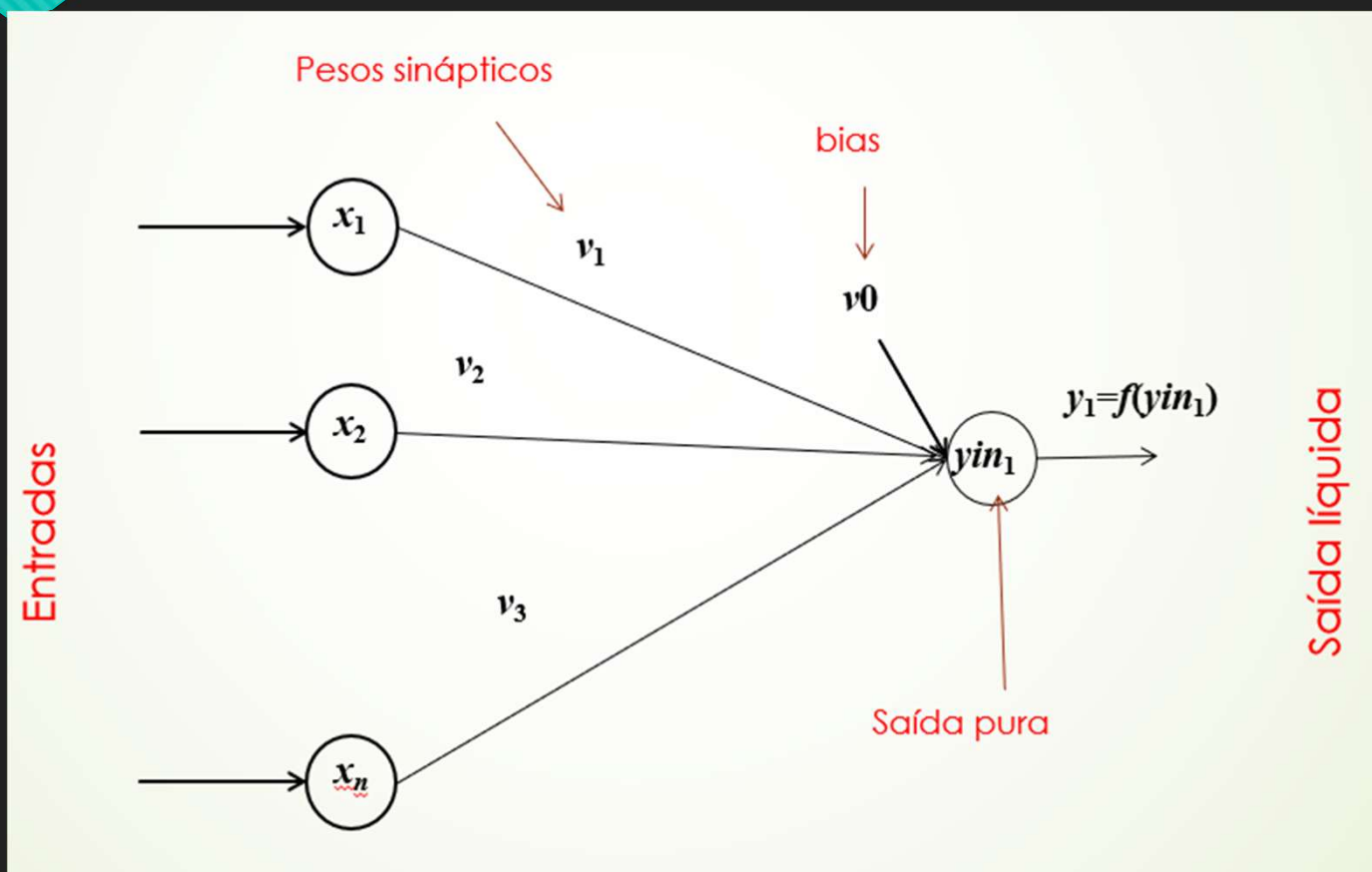
Jefferson Beethoven Martins ✓
Engenheiro de Computação - Docente Dr. - Instituto Federal de Educação, Ciência e Tecnologia do Triângulo Mineiro
Uberaba, Minas Gerais, Brasil - [Informações de contato](#)

IFTM - Instituto Federal de Educação, Ciência e Tecnologia do Triângulo Mineiro
Universidade Federal de Uberlândia

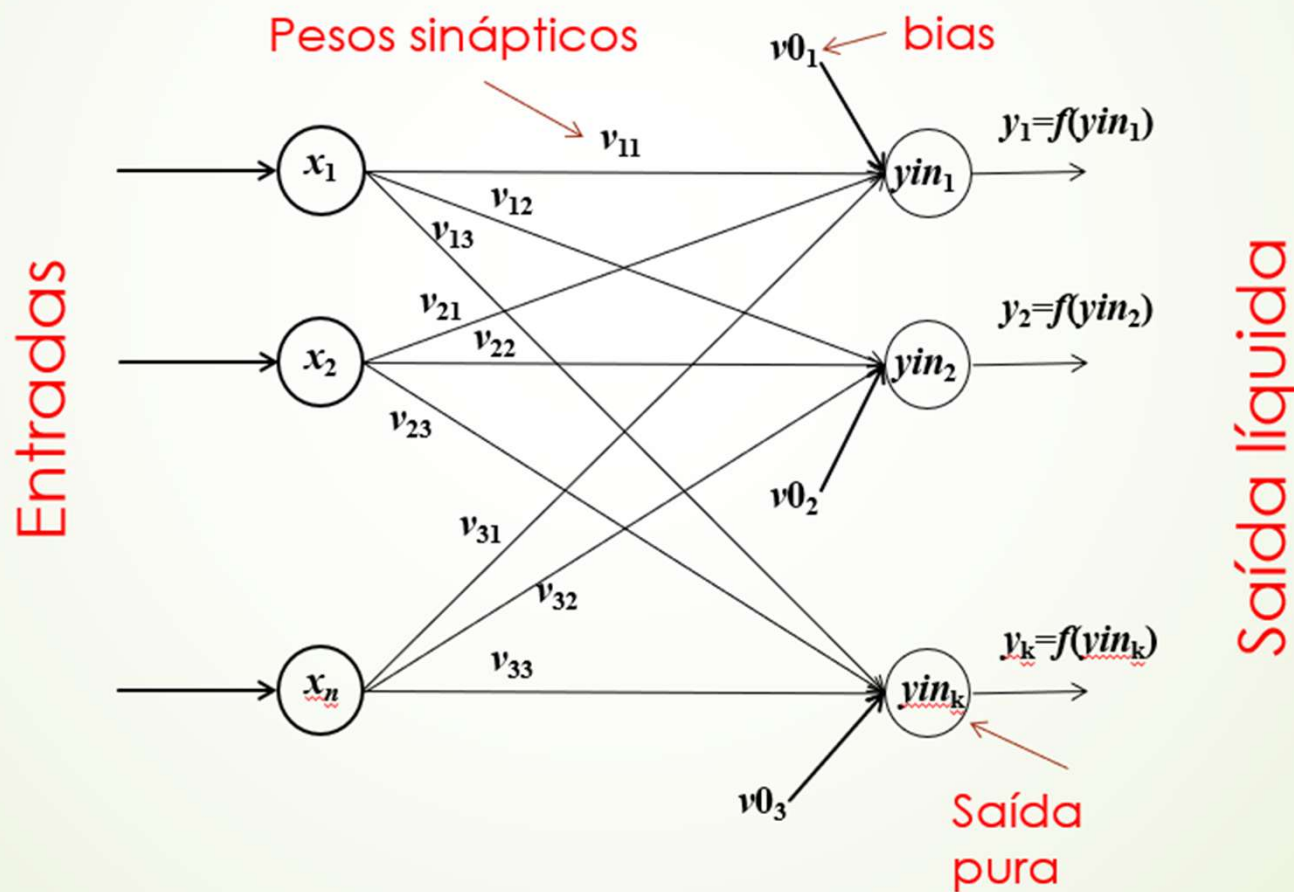
Rede Adaline

- Foi idealizada por Bernard Widrow e Marcian Hoff em 1960 (Multiple ADaptive LINear Elements).
- A aplicação principal era o chaveamento de circuitos telefônicos.
- Trata-se de uma rede de arquitetura simples, porém agregou uma nova técnica de treinamento com a utilização da regra Delta (Gradiente Descendente).
- Isso possibilitou a ampliação das aplicações em outros tipos de problemas, principalmente aqueles relacionados a sinais analógicos e consequentemente as primeiras aplicações industriais de RNAs.

Arquitetura Adaline



Arquitetura Madaline



Aspectos gerais

- O treinamento pela regra Delta também é do tipo supervisionado.
- Pertence a arquitetura feedforward.
- É comumente empregada em problemas de reconhecimento de padrões.
- Assim como ocorre com a Rede Perceptron, a Adaline é capaz de reconhecer padrões pertencentes apenas a duas classes.
- No entanto, a Madaline permite a resolução de problemas onde se tenha mais do que duas classes de padrões.

Rede Adaline ou Madaline – do ponto de vista matemático

- Em termos matemáticos, a Rede Adaline é bem semelhante a Perceptron. Quando se trata da Rede Madaline, há acréscimo do índice da saída.

$$y_k = \begin{cases} 1, & \text{se } \sum v_{ik} \cdot x_i + b \geq \text{limiar} \\ 0, & \text{se } \sum v_{ik} \cdot x_i + b < \text{limiar} \end{cases}$$

OU

$$y_k = \begin{cases} 1, & \text{se } \sum v_{ik} \cdot x_i + b \geq \text{limiar} \\ -1, & \text{se } \sum v_{ik} \cdot x_i + b < \text{limiar} \end{cases}$$

Rede Adaline ou Madaline – do ponto de vista matemático

- O treinamento pela regra Delta usa o erro para o ajuste dos pesos sinápticos.

$$e = t - y$$

Rede Adaline – Parâmetros

Parâmetro	Representação	Tipo característico
Entradas	x_i	Reais ou binárias (pode ser necessário que sejam normalizados dependendo do problema)
Pesos sinápticos	v_{ik}	Reais (aleatórios entre -0,5 e 0,5)
Bias	$v0_k$	Reais (aleatórios entre -0,5 e 0,5)
Saída pura	yin_k	Real – Resultado da adição entre o bias e a somatória dos produtos entre as entradas e seus respectivos pesos.
Função de ativação	$f(yin_k)$	Degrau ou Degrau Bipolar
Saída Líquida	y_k	Binária ou Bipolar
Processo de treinamento	-----	Supervisionado
Regra de Aprendizado	-----	Regra Delta

Rede Adaline – Fundamentação da fórmula de treinamento

- A regra Delta utiliza a aplicação do Gradiente Descendente.
- Esse método também é conhecido como algoritmo *LMS* (**L**east **M**ean **S**quare).
- O gradiente descendente é aplicado sobre a fórmula do erro para obter as fórmulas de atualização dos pesos sinápticos e do bias.

Rede Adaline – Fundamentação da fórmula de treinamento

- Considerando o erro proveniente da comparação entre as saídas geradas pela rede e as saídas esperadas (targets) é dado pela fórmula abaixo.

$$E(v) = \frac{1}{2} \sum_{i=1}^k (t^{(k)} - y)^2$$

- Substituímos a fórmula de cálculo do valor de y na expressão anterior.

$$E(v) = \frac{1}{2} \sum_{k=1}^p \left(t^{(k)} - \left(\sum v_{ip} \cdot x_i + b \right) \right)^2$$
$$E(v) = \frac{1}{2} \sum_{k=1}^p \left(t^{(k)} - \left(v^T \cdot x^{(k)} + b \right) \right)^2$$

Rede Adaline – Fundamentação da fórmula de treinamento

- A atualização dos pesos será dada por:

$$v^{novo} = v^{anterior} + \alpha \sum_{k=1}^p (t^{(k)} - y) \cdot x^{(k)}$$

- A atualização do bias será dada por:

$$v0^{novo} = v0^{anterior} + \alpha \sum_{k=1}^p (t^{(k)} - y)$$

Rede Adaline – Algor. de treinamento

1. Obter o conjunto de amostras de treinamento;
2. Associar a saída desejada $(t^{(k)})$ para cada amostra obtida;
3. Iniciar os vetores v e b com valores aleatórios pequenos (entre $-0,5$ e $0,5$);
4. Escolher um valor para a taxa de aprendizagem α e a condição de parada;
5. Iniciar o contador de ciclos (épocas);
6. Repetir as instruções:
 - 6.1 Para cada amostra de treinamento $\{x^{(k)}, t^{(k)}\}$
 - 6.2 ciclo=ciclo+1
 - 6.3 $E_{qt}=0$;
 - 6.3.1 $yin = x^k \cdot v^T + v0$;
 - 6.3.2 $y = f(yin)$;
 - 6.3.3 $E_{qt} = E_{qt} + \frac{1}{2} \sum_{i=1}^k (t^{(k)} - y)^2$
 - 6.3.4 $v^{novo} = v^{anterior} + \alpha \sum_{k=1}^P (t^{(k)} - y) \cdot x^{(k)}$
 - 6.3.5 $v0^{novo} = v0^{anterior} + \alpha \sum_{k=1}^P (t^{(k)} - y)$

Até que: a condição de parada seja satisfeita

Rede Adaline – Algoritmo de treinamento

- A condição de parada pode ser por alcance de determinado número de ciclos ou pelo alcance de um erro quadrático médio máximo.
- O valor da taxa de aprendizagem deve estar compreendido entre 0 e 1.
- Deve-se ter cuidado para o valor escolhido para a taxa de aprendizagem mesmo dentro do intervalo indicado.
- Um valor alto pode causar instabilidade na convergência do erro. Um valor extremamente baixo pode fazer com que a convergência seja muito lenta.

Regra One-of-Classes

- Até o momento trabalhamos com projetos em que havia apenas uma saída para a RNA.
- Redes Madaline podem ser utilizadas para resolver problemas com mais do que duas classes.
- Para resolver problemas dessa natureza, utiliza-se a regra One-of-Classes, que consiste em criar vetores com tamanho igual ao número de classes, em que a posição do componente 1 indica a classe do problema.
- O número de saídas da rede está diretamente ligado ao número de classes do problema. Se por exemplo a RNA for utilizada para reconhecer a foto de 4 pessoas, a rede deverá ter 4 saídas. E assim por diante...

Regra One-of-Classes – 5 classes

Classe 1	Classe 2	Classe 3	Classe 4	Classe 5
1	0	0	0	0
0	1	0	0	0
0	0	1	0	0
0	0	0	1	0
0	0	0	0	1

Classe 1	Classe 2	Classe 3	Classe 4	Classe 5
1	-1	-1	-1	-1
-1	1	-1	-1	-1
-1	-1	1	-1	-1
-1	-1	-1	1	-1
-1	-1	-1	-1	1

Eita... É Python

```
C:\Exemplo_Madeline\Madaline.py

temp.py × Madaline.py ×

1  # -*- coding: utf-8 -*-
2  # Autor: Jefferson Beethoven Martins - adaptado de José Ricardo Gonçalves Manzan.
3  import os
4  import numpy as np
5  import random as rd
6
7  print("\x1b[2J\x1b[1;1H")
8  os.chdir(r'C:\Exemplo_Madeline')
9
10 entAux = np.loadtxt('Ent.txt')
11 (padroes, entradas) = np.shape(entAux)
12
13 targAux = np.loadtxt('Targ.csv', delimiter=';', skiprows=0)
14 (numSaidas, targets) = np.shape(targAux)
15
16 limiar = 0.0
17 alfa = 0.01
18 errotolerado = 0.01
19 v = np.zeros((entradas, numSaidas))
20 for i in range(entradas):
21     for j in range(numSaidas):
22         v[i][j] = rd.uniform(-0.1, 0.1)
```

C:\Exemplo_Madaline\Madaline.py



temp.py ×

Madaline.py ×

```
19 v = np.zeros((entradas, numSaidas))
20 for i in range(entradas):
21     for j in range(numSaidas):
22         v[i][j] = rd.uniform(-0.1, 0.1)
23
24 v0 = np.zeros((numSaidas))
25 for j in range(numSaidas):
26     v0[j] = rd.uniform(-0.1, 0.1)
27
28 yin = np.zeros((numSaidas))
29 y = np.zeros((numSaidas))
30
31 erro = 1.0
32 ciclo = 0
33
34 while erro > errotolerado and ciclo < 10000:
35     ciclo = ciclo + 1
36     erro = 0
37     for i in range(padroes):
38         padraoLetra = entAux[i, :]
39
40         for m in range(numSaidas):
41             soma = 0
42             for n in range(entradas):
43                 soma = soma + padraoLetra[n] * v[n][m]
44             yin[m] = soma + v0[m]
45
```

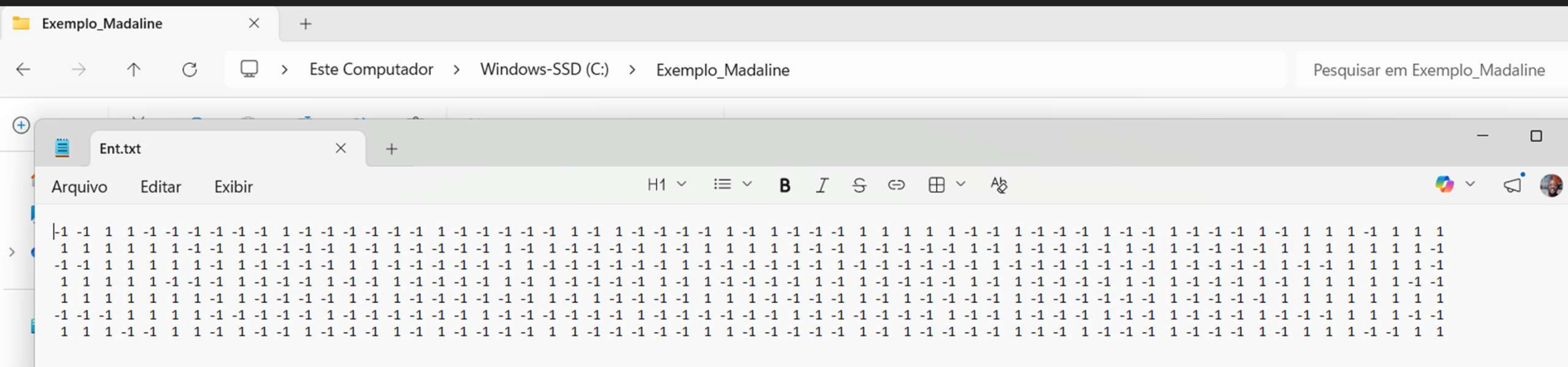


```
45     for j in range(numSaidas):
46         if yin[j] >= limiar:
47             y[j] = 1.0
48         else:
49             y[j] = -1.0
50
51     for j in range(numSaidas):
52         erro = erro + 0.5 * ((targAux[j][i] - y[j])**2)
53
54     vanterior = v.copy()
55     v0anterior = v0.copy()
56
57     for m in range(entradas):
58         for n in range(numSaidas):
59             v[m][n] = vanterior[m][n] + alfa * (targAux[n][i] - y[n]) * padraoLetra[m]
60
61     for j in range(numSaidas):
62         v0[j] = v0anterior[j] + alfa * (targAux[j][i] - y[j])
63
64     print(f"Ciclo: {ciclo} - Erro: {erro}")
65
66 # Rede treinada com sucesso.
67 entTeste = entAux[6, :]
68 for j in range(numSaidas):
69     soma = 0
70     for i in range(entradas):
71         soma = soma + entTeste[i] * v[i][j]
72     yin[j] = soma + v0[j]
```

Eita... É Python

```
66
67 # Rede treinada com sucesso.
68 entTeste = entAux[6, :]
69 for j in range(numSaidas):
70     soma = 0
71     for i in range(entradas):
72         soma = soma + entTeste[i] * v[i][j]
73     yin[j] = soma + v0[j]
74
75 print("Yin final:", yin)
76 for j in range(numSaidas):
77     if yin[j] >= limiar:
78         y[j] = 1.0
79     else:
80         y[j] = -1.0
81 print("Y final:", y)
```

Quem é o arquivo de entrada?



Exemplo_Madaline

Este Computador > Windows-SSD (C:) > Exemplo_Madaline

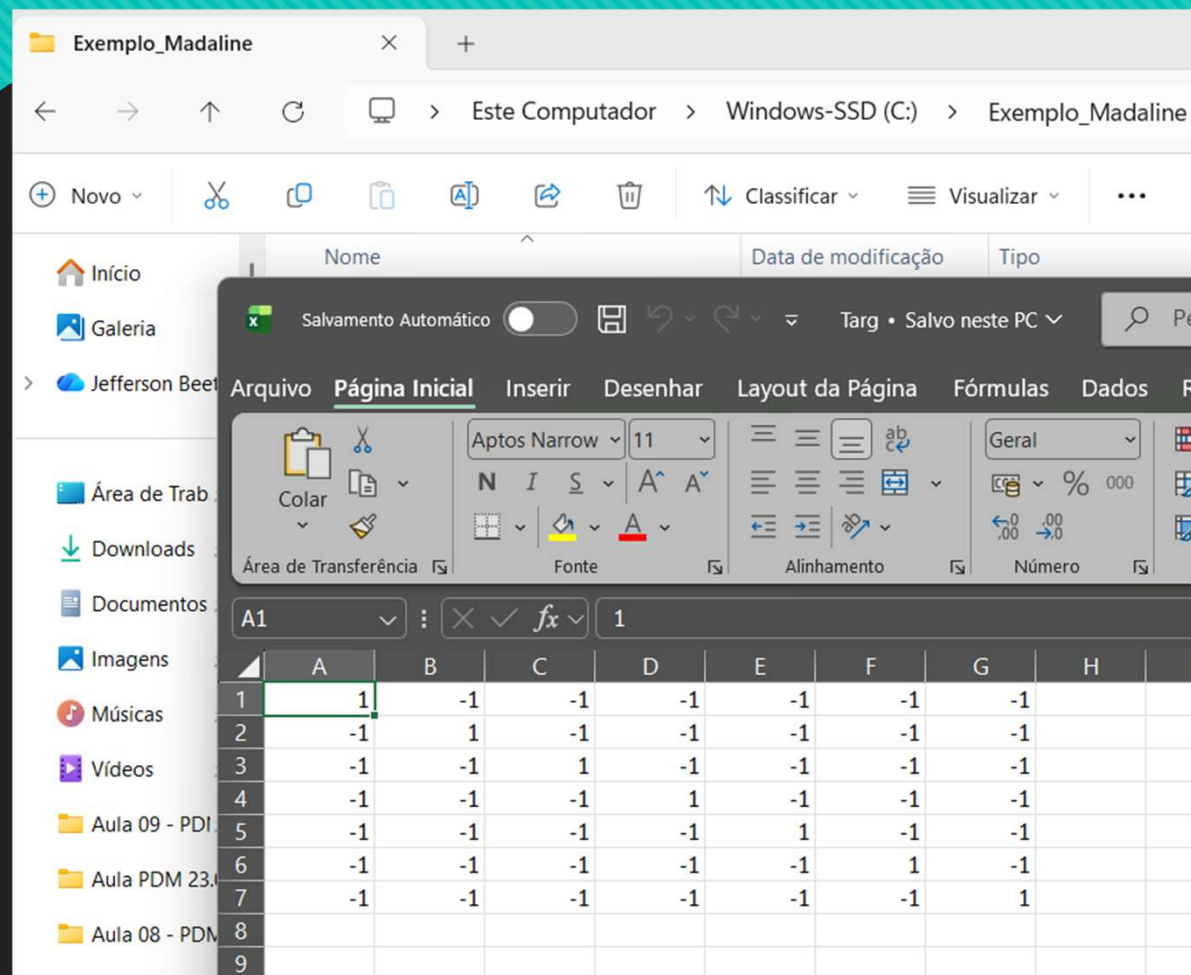
Pesquisar em Exemplo_Madaline

Ent.txt

Arquivo Editar Exibir

H1 B I S ↺ ↻

```
| 1 -1 1 1 -1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 1 1 1 1 -1 -1 1 -1 -1 -1 1 -1 -1 -1 1 1 1 1 -1  
1 1 1 1 1 1 -1 -1 1 -1 -1 -1 1 -1 1 -1 -1 -1 1 -1 1 -1 -1 -1 1 1 1 1 -1 -1 1 -1 -1 -1 1 1 1 1 1 1 1 1 1 -1  
-1 -1 1 1 1 1 1 -1 1 -1 -1 -1 1 1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1 1 1 1 1 1 -1  
1 1 1 1 1 1 -1 -1 1 -1 -1 -1 1 -1 -1 -1 -1 1 -1 1 -1 -1 -1 -1 1 -1 1 -1 -1 -1 1 -1 1 -1 -1 -1 1 1 1 1 1 -1 -1  
1 1 1 1 1 1 1 -1 1 -1 -1 -1 1 -1 1 -1 -1 -1 1 1 1 -1 -1 -1 1 -1 1 -1 -1 -1 1 -1 -1 -1 -1 1 1 1 1 1 1 1 1  
-1 -1 -1 1 1 1 1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1 1 -1 -1 -1 -1 -1 1 1 1 1 1 -1  
1 1 1 -1 -1 1 1 -1 1 -1 -1 1 -1 -1 -1 -1 1 1 -1 -1 -1 -1 1 1 -1 -1 -1 -1 1 -1 -1 -1 -1 1 1 1 -1 -1 1 1
```

Mais um trabalho (trabalho 03)

- Faça um programa que reconheça as 13 letras do alfabeto por meio de um algoritmo Madaline. O usuário deve construir as letras em uma interface visual (CheckBoxes).

